

Technische Dokumentation

Repository: galabaustreich-astro
Branch: feat/astro-modernisierung
Commit: 3c837baf80be192e4c782180ccb951126dce9878
Generiert: 2026-09-09T18:11:01.553Z
Facts: 1234 · Relationships: 280
Modus: evidence-first
Provider: ollama / qwen3.5:9b

Inhaltsverzeichnis

Technische Dokumentation

Einstieg

Dokumentenindex

1. Projektübersicht

Erklärung

Kurzreferenz

2. Systemarchitektur

Erklärung

Kurzreferenz

3. Technologie-Stack

Erklärung

Kurzreferenz

4. Repository-Struktur

Erklärung

Kurzreferenz

5. Komponenten und Verantwortlichkeiten

Erklärung

Kurzreferenz

6. Datenflüsse

Erklärung

Kurzreferenz

7. APIs und Schnittstellen

Erklärung

Kurzreferenz

8. Datenhaltung

Erklärung

Kurzreferenz

9. Konfiguration

Erklärung

Kurzreferenz

10. Dependencies

Erklärung

Kurzreferenz

11. Build und lokale Entwicklung

Erklärung

Kurzreferenz

12. Deployment und CI/CD

Erklärung

Kurzreferenz

13. Tests und Qualitätssicherung

Erklärung

Kurzreferenz

14. Security-relevante Architektur

Erklärung

Kurzreferenz

15. belegbare Architekturentscheidungen

Erklärung

Kurzreferenz

16. bekannte Einschränkungen

Erklärung

Kurzreferenz

17. belegbarer Technical Debt

Erklärung

Kurzreferenz

18. Git-/Änderungsverlauf

Erklärung

Kurzreferenz

19. aktueller technischer Zustand

Erklärung

Kurzreferenz

Technische Dokumentation

- Schema: techdoc-schema:v2-evidence
- Modus: evidence-first + narrative
- Dokumentierter Commit: 3c837baf80be192e4c782180ccb951126dce9878
- Branch: feat/astro-modernisierung
- Facts: 1234
- Relationships: 280

> Quelle der Wahrheit: Git-Repository-Analyse (Evidence Graph). Die KI erklärt und verbindet nur belegte Fakten.

Einstieg

Das Repository für das Projekt „Projektübersicht“ befindet sich aktuell im Branch feat/astro-modernisierung und ist als moderne Astro-Anwendung strukturiert, die auf Netlify bereitgestellt wird. Der Build-Prozess wird durch den Befehl `npm run build` in der Datei `netlify.toml` ausgelöst, wobei das resultierende Ausgabeverzeichnis `dist` direkt an Netlify gepusht wird. Für die Bündelung des Frontends steht `esbuild` als Node-Bundler zur Verfügung. Die Anwendung folgt einer klaren Komponentenarchitektur: Das Layout wird durch `BaseLayout` und den speziellen `LegalLayout` definiert, während UI-Elemente wie Header, Footer, Logo, Icon sowie spezifische Sektionen wie `ContactCTA`, `ContactForm`, `FloatingWhatsApp`, `SeasonalServices` und `Rating` modular aufgebaut sind. Die Seitenstruktur umfasst die Startseite (`index`), Unterseiten für Leistungen (`leistungen`) und Team (`team`) sowie rechtliche Pflichtseiten wie `agb`, `datenschutz` und `impressum`. Zusätzlich existiert eine eigene Fehlerseite (`404`).

Die Sicherheitskonfiguration ist in der `netlify.toml` detailliert festgelegt. Für alle Ressourcen (`/*`) werden die Header `X-Content-Type-Options` mit dem Wert `nosniff`, `X-Frame-Options` mit `DENY` sowie `Referrer-Policy` auf `strict-origin-when-cross-origin` gesetzt, um Clickjacking und MIME-Typ-Missbrauch zu verhindern. Zudem wird die Nutzung sensibler Gerätefunktionen wie Kamera, Mikrofon und Geolokalisierung durch das Header `Permissions-Policy` explizit blockiert (`camera=()`, `microphone=()`, `geolocation=()`). Für Font-Ressourcen unter dem Pfad `/fonts/*` ist eine aggressive Caching-Strategie konfiguriert (`Cache-Control: public, max-age=31536000, immutable`), um die Performance zu optimieren. `<!-- not-evidenced: MIME -->`

Die Interaktion mit dem Backend erfolgt über einen spezifischen Endpunkt für Kontaktanfragen. Die Funktion `contact.mjs` im Verzeichnis `frontend/netlify/functions` greift auf den Route-Pfad `/api/contact` zu und erlaubt ausschließlich HTTP-POST-Anfragen; alle anderen Methoden werden mit einem 405-Statuscode abgewiesen. Wenn ein Formular (`ContactForm`) übermittelt wird, wird diese Anfrage an diesen Endpunkt gesendet. Im Hintergrund verarbeitet das Python-Skript `backend/mailer.py`, das über `server.py` erreichbar ist, die eingehenden Daten und leitet sie per SMTP (unterstützt durch `Nodemailer`) weiter, um E-Mails zu versenden. Zusätzlich gibt es eine Cron-Job-Infrastruktur im Ordner `.emergent/cron`, die Webhooks überwacht und dispatchet, was auf eine geplante Automatisierung hindeutet.

Dokumentenindex

Dokument	Inhalt
[ARCHITECTURE.md](./ARCHITECTURE.md)	Komponenten, Routes, Beziehungen
[REPOSITORY_MAP.md](./REPOSITORY_MAP.md)	Dateien & Verzeichnisse
[DATA_FLOWS.md](./DATA_FLOWS.md)	HTTP, Mail, Flow-Ketten
[CONFIGURATION.md](./CONFIGURATION.md)	Env, Redirects, Inkonsistenzen
[DEPLOYMENT.md](./DEPLOYMENT.md)	Build, Publish, Functions
[SECURITY.md](./SECURITY.md)	Limits, Honeypot, TLS
[TESTING.md](./TESTING.md)	Tests
[DEPENDENCIES.md](./DEPENDENCIES.md)	Dependencies & Imports
[GIT_HISTORY.md](./GIT_HISTORY.md)	HEAD, Notes, Inkonsistenzen
[evidence.json](./evidence.json)	Maschinenlesbarer Evidence Graph
components/	Komponenten-Detail
api/	API-/Function-Detail

1. Projektübersicht

Erklärung

Das Repository für das Projekt „Projektübersicht“ befindet sich aktuell im Branch feat/astro-modernisierung und ist als moderne Astro-Anwendung strukturiert, die auf Netlify bereitgestellt wird. Der Build-Prozess wird durch den Befehl `npm run build` in der Datei `netlify.toml` ausgelöst, wobei das resultierende Ausgabeverzeichnis `dist` direkt an Netlify gepusht wird. Für die Bündelung des Frontends steht `esbuild` als Node-Bundler zur Verfügung. Die Anwendung folgt einer klaren Komponentenarchitektur: Das Layout wird durch `BaseLayout` und den speziellen `LegalLayout` definiert, während UI-Elemente wie Header, Footer, Logo, Icon sowie spezifische Sektionen wie `ContactCTA`, `ContactForm`, `FloatingWhatsApp`, `SeasonalServices` und `Rating` modular aufgebaut sind. Die Seitenstruktur umfasst die Startseite (`index`), Unterseiten für Leistungen (`leistungen`) und Team (`team`) sowie rechtliche Pflichtseiten wie `agb`, `datenschutz`

und impressum. Zusätzlich existiert eine eigene Fehlerseite (404).

Die Sicherheitskonfiguration ist in der `netlify.toml` detailliert festgelegt. Für alle Ressourcen (`/*`) werden die Header `X-Content-Type-Options` mit dem Wert `nosniff`, `X-Frame-Options` mit `DENY` sowie `Referrer-Policy` auf `strict-origin-when-cross-origin` gesetzt, um Clickjacking und MIME-Typ-Missbrauch zu verhindern. Zudem wird die Nutzung sensibler Gerätefunktionen wie Kamera, Mikrofon und Geolokalisierung durch das Header `Permissions-Policy` explizit blockiert (`camera=()`, `microphone=()`, `geolocation=()`). Für Font-Ressourcen unter dem Pfad `/fonts/*` ist eine aggressive Caching-Strategie konfiguriert (`Cache-Control: public, max-age=31536000, immutable`), um die Performance zu optimieren. `<!-- not-evidenced: MIME -->`

Die Interaktion mit dem Backend erfolgt über einen spezifischen Endpunkt für Kontaktanfragen. Die Funktion `contact.mjs` im Verzeichnis `frontend/netlify/functions` greift auf den Route-Pfad `/api/contact` zu und erlaubt ausschließlich HTTP-POST-Anfragen; alle anderen Methoden werden mit einem 405-Statuscode abgewiesen. Wenn ein Formular (`ContactForm`) übermittelt wird, wird diese Anfrage an diesen Endpunkt gesendet. Im Hintergrund verarbeitet das Python-Skript `backend/mailer.py`, das über `server.py` erreichbar ist, die eingehenden Daten und leitet sie per SMTP (unterstützt durch `Nodemailer`) weiter, um E-Mails zu versenden. Zusätzlich gibt es eine Cron-Job-Infrastruktur im Ordner `.emergent/cron`, die Webhooks überwacht und dispatchet, was auf eine geplante Automatisierung hindeutet.

Kurzreferenz

- Header `X-Content-Type-Options` for `/*`: `{"for": "/*", "header": "X-Content-Type-Options", "value": "nosniff", "cache": false}` · `netlify.toml:46` — `X-Content-Type-Options = "nosniff" for /*`
- Header `X-Frame-Options` for `/*`: `{"for": "/*", "header": "X-Frame-Options", "value": "DENY", "cache": false}` · `netlify.toml:47` — `X-Frame-Options = "DENY" for /*`
- Header `Referrer-Policy` for `/*`: `{"for": "/*", "header": "Referrer-Policy", "value": "strict-origin-when-cross-origin", "cache": false}` · `netlify.toml:48` — `Referrer-Policy = "strict-origin-when-cross-origin" for /*`
- Header `Permissions-Policy` for `/*`: `{"for": "/*", "header": "Permissions-Policy", "value": "camera=(), microphone=(), geolocation=()", "cache": false}` · `netlify.toml:49` — `Permissions-Policy = "camera=(), microphone=(), geolocation=()" for /*`
- Header `Cache-Control` for `/fonts/*`: `{"for": "/fonts/*", "header": "Cache-Control", "value": "public, max-age=31536000, immutable", "cache": true}` · `netlify.toml:54` — `Cache-Control = "public, max-age=31536000, immutable" for /fonts/*`

2. Systemarchitektur

Erklärung

Die Systemarchitektur dieses Repositories basiert auf einer modernen, serverlosen Frontend-Architektur mit Astro als Framework und Netlify als Hosting-Plattform. Das Frontend wird primär durch eine Sammlung von Astro-Komponenten definiert, die in spezifischen Verzeichnissen organisiert sind. Die Layouts bilden das strukturelle Gerüst der Anwendung: `BaseLayout` dient als universeller Ausgangspunkt für den Großteil der Seiten, während `LegalLayout` speziell für rechtliche Dokumente wie AGB (`agb.astro`), Datenschutz (`datenschutz.astro`) und Impressum (`impressum.astro`) verwendet wird. Für die Darstellung von Inhalten stehen zahlreiche wiederverwendbare Komponenten zur Verfügung, darunter Header, Footer, Logo, Icon sowie domänenspezifische Elemente wie `ContactCTA`, `ContactForm`, `FloatingWhatsApp`, `SeasonalServices` und `LegalSection`. Die Routenstruktur ist klar getrennt: Standardseiten wie die Startseite (`index.astro`) oder das Team (`team.astro`) nutzen das Basis-Layout, während rechtliche Seiten das s

Die Backend-Kommunikation und API-Logik laufen vollständig serverlos über Netlify Functions. Ein zentraler Mechanismus ist die globale Rewrite-Regel in `netlify.toml`, die alle Anfragen unter `/api/*` auf die generische Funktion `/.netlify/functions/:splat` weiterleitet, was eine zentrale Verwaltung von API-Endpunkten ermöglicht. Der spezifische Endpunkt für Kontaktanfragen ist `/api/contact`. Dieser wird durch die Datei `frontend/netlify/functions/contact.mjs` implementiert. Die Architektur erzwingt hier eine strikte Methodentrennung: Das Serverless-Funktionsskript erlaubt explizit nur HTTP-POST-Anfragen und lehnt alle anderen Methoden mit dem Statuscode 405 ab, was auf einer Guard-Logik innerhalb der Funktion basiert.

Der Datenfluss bei Kontaktanfragen ist klar definiert und verbindet Frontend, Backend-Logik und externe Dienste. Wenn ein Nutzer das Formular `ContactForm` ausfüllt, wird die Anfrage als POST an `/api/contact` gesendet. Im Hintergrund verarbeitet die Serverless-Funktion diese Eingabe und leitet sie weiter an einen Python-basierten Mailer (`backend/mailer.py`). Dieser Backend-Teil ist für sich genommen eine separate Komponente mit eigenen Abhängigkeiten (`requirements.txt`) und Testkonfigurationen (`pytest.ini`). Die eigentliche E-Mail-Versendung erfolgt über SMTP, gesteuert durch den `backend/mailer.py` Code. Zusätzlich existiert im Repository eine eigene Cron-Job-Infrastruktur unter `.emergent/cron`, die Webhooks überwacht und auslöst (`.emergent/cron/watch_crons.sh`, `.emergent/cron/dispatch_webhook.sh`). Diese Infrastruktur ist jedoch aktuell noch nicht direkt mit dem Kontaktformular-Workflow verknüpft, sondern stellt eine separate Automatisierungsebene dar. Die Architektur trennt somit die reine Fro

Kurzreferenz

- `/api/* 200 /.netlify/functions/:splat: rewrite_897 !' /.netlify/functions/:splat`
- `GET: srcfile_9 !' /api/contact`
- `sends_mail: srcfile_15 !' smtp_57`
- `posts_to: form_88 !' /api/contact`
- `POST: srcfile_446 !' /api/contact`
- `contact.ts !' Nodemailer/SMTP: serverless_46 !' smtp_57`

3. Technologie-Stack

Erklärung

Abhängigkeiten stammen aus `package.json`; tatsächliche Nutzung zeigt sich in `Import-Facts`.

Keine `Dependencies` belegt.

Beobachtete Imports (Auszug): `frontend/astro.config.mjs !' astro/config; frontend/astro.config.mjs !' node:fs/promises; frontend/netlify/functions/contact.mjs !' nodemailer; frontend/playwright.config.ts !' @playwright/test; frontend/src/components/ContactCTA.astro !' ../data/site; frontend/src/components/ContactCTA.astro !' ../Icon.astro; frontend/src/components/ContactForm.astro !' ../data/site; frontend/src/components/ContactForm.astro !' ../Icon.astro; frontend/src/components/FloatingWhatsApp.astro !' ../data/site; frontend/src/components/FloatingWhatsApp.astro !' ../Icon.astro; frontend/src/components/Footer.astro !' ../data/site; frontend/src/components/Footer.astro !' ../Logo.astro.`

Kurzreferenz

`_` Nicht dokumentiert — keine belegten Einträge.`_`

4. Repository-Struktur

Erklärung

Dieses Repository ist eine belegbare Web-/Deploy-Codebasis.

Eine Site-URL ist in den extrahierten Config-Fakten nicht belegt.

Die statische Analyse hat 1234 technische Fakten und 280 Beziehungen erfasst, darunter 20 Komponenten-Fakten und 2 Endpoint-Hinweise.

Die folgenden Abschnitte und Detail-Dateien beschreiben Aufbau, Datenflüsse, APIs, Deployment und Security so, dass ein neuer Entwickler die Laufzeitpfade nachvollziehen kann, ohne das gesamte Repository zuerst selbst zu durchsuchen.

Aussagen ohne Evidence-Pfad sind bewusst als nicht dokumentiert markiert; es werden keine Architekturmotive oder Technical-Debt-Urteile spekuliert.

Kurzreferenz

Siehe `REPOSITORY_MAP.md`. 781 Dateien, 145 Verzeichnisse.

5. Komponenten und Verantwortlichkeiten

Erklärung

Die Komponente `InquiryPanel` ist ein zentraler Kontaktmechanismus innerhalb der Astro-basierten Frontend-Architektur, der primär durch das Formular-Element `ContactForm.astro` realisiert wird. Dieses Formular leitet alle Benutzereingaben an den Serverless-Endpoint `/api/contact` weiter, wobei die HTTP-Methode strikt auf `POST` beschränkt ist; Versuche, andere Methoden zu verwenden, werden vom Backend mit einem 405-Fehler abgewiesen. Das Formular selbst definiert eine spezifische Struktur aus Pflichtfeldern wie dem Namen (maximal 120 Zeichen) und der Nachricht (bis zu 4000 Zeichen), sowie optionalen Feldern für Telefon, E-Mail, Standort und Website. Ein besonderes Sicherheitsmerkmal ist das Feld `website`, das als „Honeypot“ konfiguriert ist, um automatisierte Bots zu erkennen, während das Feld `consent` eine zwingende Einwilligungserklärung erfordert.

Im Hintergrund wird die Verarbeitung dieser Daten durch eine Netlify Function (`contact.mjs`) gesteuert, die den `POST`-Anfang empfängt und anstelle einer direkten Datenbankpersistierung oder komplexen Logik primär eine E-Mail-Übermittlung initiiert. Dieser Prozess greift auf das Backend-Modul `backend/mailer.py` zurück, welches über SMTP (implementiert in `server.py`) versendet wird. Die Architektur trennt hier klar die Präsentationsschicht im Frontend von der Logikschicht in den Netlify Functions und dem Python-Backend, wobei die Datenübertragung ausschließlich als JSON-Payload erfolgt.

Die Integration des `InquiryPanel` erfolgt typischerweise innerhalb einer `BaseLayout` oder spezieller Layouts wie `LegalLayout`, was eine konsistente Darstellung über verschiedene Seiten hinweg (z. B. Impressum oder AGB) ermöglicht. Zusätzlich zum direkten Formular gibt es oft einen Call-to-Action (`ContactCTA`) und ein

schwebendes WhatsApp-Fenster (FloatingWhatsApp), die als alternative Kontaktwege dienen, aber technisch unabhängig vom Hauptformular laufen. Die gesamte Implementierung befindet sich aktuell im Branch feat/astro-modernisierung, was auf eine aktive Weiterentwicklung oder Migration hinweist, wobei alle genannten Komponenten und Endpunkte bereits in der aktuellen Repository-Struktur verortet sind.

Kurzreferenz

- [ContactCTA.astro](./components/ContactCTA.md)
- [ContactForm.astro](./components/ContactForm.md)
- [FloatingWhatsApp.astro](./components/FloatingWhatsApp.md)
- [Footer.astro](./components/Footer.md)
- [Header.astro](./components/Header.md)
- [Icon.astro](./components/Icon.md)
- [LegalSection.astro](./components/LegalSection.md)
- [Logo.astro](./components/Logo.md)
- [Rating.astro](./components/Rating.md)
- [SeasonalServices.astro](./components/SeasonalServices.md)
- [BaseLayout.astro](./components/BaseLayout.md)
- [LegalLayout.astro](./components/LegalLayout.md)
- [404.astro](./components/404.md)
- [agb.astro](./components/agb.md)
- [datenschutz.astro](./components/datenschutz.md)
- [impressum.astro](./components/impressum.md)
- [index.astro](./components/index.md)
- [leistungen.astro](./components/leistungen.md)
- [team.astro](./components/team.md)
- [ueber-uns.astro](./components/ueber-uns.md)

6. Datenflüsse

Erklärung

Der Datenfluss im Repository beginnt auf der Frontend-Seite, wo das Formular-Element in frontend/src/components/ContactForm.astro definiert ist und sein action-Attribut explizit auf /api/contact verweist. Wenn ein Nutzer das Formular absendet, wird eine HTTP-POST-Anfrage an diesen Pfad gerichtet; diese Anfrage wird intern von der Datei frontend/netlify/functions/contact.mjs verarbeitet, die als Implementierung für den Endpunkt /api/contact dient. Wichtig ist hierbei die Konfiguration in netlify.toml, die sicherstellt, dass alle Requests unter /api/* automatisch an die Serverless-Funktion /.netlify/functions/:splat weitergeleitet werden, wobei diese Weiterleitung mit einem HTTP-Statuscode 200 bestätigt wird. Die Logik innerhalb der Funktion contact.mjs greift auf Umgebungsvariablen wie SMTP_HOST, SMTP_PORT, SMTP_USER und SMTP_PASSWORD zu, um eine Verbindung zum E-Mail-Server herzustellen, wobei die TLS-Verschlüsselung aktiviert ist.

Sobald die Anfrage validiert wurde (die Funktion erlaubt nur POST-Anfragen und lehnt andere Methoden mit Status 405 ab), initiiert der Code den Versand einer E-Mail vom Typ „inquiry“. Diese Nachricht wird an den Empfänger gesendet, dessen Adresse entweder aus der Konfiguration CONTACT_RECIPIENT oder direkt aus der Umgebungsvariable SMTP_USER stammt. Parallel dazu wird eine Bestätigungsmail des Typs „confirmation“ generiert und versendet. Der gesamte E-Mail-Versandprozess nutzt die Bibliothek nodemailer, um die SMTP-Kommunikation zu handhaben, wobei die Datenflüsse hier klar vom Frontend-Code über die Serverless-Funktion zum externen SMTP-Server fließen.

Neben diesem direkten Benutzerinteraktionspfad existiert im Repository eine separate Infrastruktur für Cron-Jobs unter .emergent/cron. Diese Verzeichnisstruktur enthält Skripte wie watch_crons.sh und dispatch_webhook.sh, die in einer Konfigurationsdatei webhook-crons definiert sind. Obwohl diese Dateien als Quellen für automatisierte Aufgaben identifiziert wurden, ist im vorliegenden Kontext nicht dokumentiert, ob oder wie diese Cron-Jobs mit dem oben beschriebenen Kontaktformular-Fluss interagieren; sie stehen isoliert als Quelle für Webhook-Auslöser bereit. Ebenso finden sich Backend-Komponenten wie backend/mailer.py und backend/server.py, die jedoch im aktuellen Datenfluss-Diagramm nicht mit den spezifischen Endpunkten oder Funktionen des Frontend-Branche verknüpft sind, sodass ihre Rolle hier als nicht belegt gilt.

Kurzreferenz

Siehe DATA_FLOWS.md.

7. APIs und Schnittstellen

Erklärung

Die API-Route `/api/contact` ist eine Serverless-Funktion, die im Repository unter `frontend/netlify/functions/contact.mjs` implementiert ist und ausschließlich über HTTP POST-Anfragen bedient wird; andere Methoden werden mit einem 405-Status abgelehnt. Der Endpunkt erwartet ein JSON-Objekt als Payload, da Form-Daten oder Base64-Codierung nicht unterstützt sind. Bevor die Logik ausgeführt wird, greift eine Größenbeschränkung von maximal 20.000 Bytes auf der Request-Ebene; überschreitet das eingehende Paket diese Grenze, wird sofort ein HTTP 413 (Payload Too Large) zurückgegeben. Die Validierung der einzelnen Felder erfolgt anhand definierter Längenlimits: `name` darf maximal 120 Zeichen, `phone` 40 Zeichen, `email` 160 Zeichen, `location` 160 Zeichen, `message` 4000 Zeichen, `page` 200 Zeichen und `website` 300 Zeichen enthalten. Besonders zu beachten ist das Feld `website`, das als Honeypot fungiert; wenn dieses Feld ausgefüllt wird, führt dies nicht zu einem stillen Erfolg, sondern löst eine Fehlerbehebung aus.

Bei erfolgreicher Validierung und Einhaltung der Limits wird eine E-Mail-Versandroutine ausgelöst, die auf dem SMTP-Protokoll basiert und die Bibliothek `nodemailer` nutzt. Die Verbindung zum Mailserver erfolgt über TLS mit einer minimalen Versionspezifikation, wobei die Konfigurationswerte für Host, Port, Benutzername und Passwort aus den Umgebungsvariablen `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER` und `SMTP_PASSWORD` gelesen werden. Das System sendet zwei verschiedene E-Mails: eine Bestätigungsmail an den Absender sowie eine Anfrage-Mail an den Empfänger, der durch die Variable `CONTACT_TO` (oder im Code als `CONTACT_RECIPIENT/SMTP_USER`) definiert ist. Sollte der SMTP-Server nicht erreichbar sein oder ein Timeout auftreten, wird ein HTTP 503 (Service Unavailable) zurückgegeben; bei Verbindungsfehlern oder falschen Serverantworten erfolgt die Rückgabe eines HTTP 502 (Bad Gateway). Ein ungültiges JSON oder fehlende Pflichtfelder führen zu einem HTTP 400 (Bad Request). Im Falle einer erfolgreichen Verarbeitung

Kurzreferenz

- HTTP method guard POST in `frontend/netlify/functions/contact.mjs` · `impl=undefined`
- `/api/contact` · `impl=frontend/netlify/functions/contact.mjs`

8. Datenhaltung

Erklärung

Persistente Anwendungsdatenhaltung ist in den extrahierten Facts nicht als eigene Datenbankschicht belegt; Kontaktfluss speichert transient und versendet per SMTP.

Kurzreferenz

Nicht dokumentiert — keine belegten Einträge.

9. Konfiguration

Erklärung

Die Konfiguration in diesem Repository ist stark durch die Nutzung von Netlify und spezifischen Astro-Plugins geprägt, wobei Sicherheitsrichtlinien und Routing-Logik zentral gesteuert werden. Im Kern der Deployment-Konfiguration befindet sich die Datei `netlify.toml`, welche die HTTP-Antworten für alle Requests (/*) standardisiert absichert. Dort sind explizit die Header `X-Content-Type-Options` mit dem Wert `nosniff`, `X-Frame-Options` auf `DENY` sowie `Referrer-Policy` auf `strict-origin-when-cross-origin` definiert, um Cross-Site-Scripting und andere Angriffe zu verhindern. Zusätzlich wird die Nutzung sensibler APIs wie Kamera, Mikrofon und Geolokation über den Header `Permissions-Policy` blockiert (`camera=()`, `microphone=()`, `geolocation=()`). Für statische Assets, insbesondere Schriftarten unter dem Pfad `/fonts/*`, ist eine aggressive Caching-Strategie konfiguriert: Der Header `Cache-Control` erlaubt ein öffentliches Caching mit einer Gültigkeitsdauer von einem Jahr (`max-age=31536000`) und der Kennzeich

Das Routing wird in derselben `netlify.toml` durch Rewrite-Regeln und Redirects definiert. Alle Anfragen unter `/api/*` werden transparent an die Serverless-Funktionen weitergeleitet, wobei der Pfadteil hinter dem Slash als Parameter für die Funktion `/.netlify/functions/:splat` genutzt wird und eine Statuscode 200 zurückgibt. Fehlt ein passender Treffer im Frontend oder in den Funktionen, leitet das System standardmäßig auf `/404.html` weiter, was durch einen Redirect mit dem Status 404 realisiert wird. Diese Struktur ermöglicht es, die API-Endpunkte dynamisch zu erweitern, ohne dass diese im Frontend-Code explizit als statische Pfade gehärtet werden müssen.

Die Umgebungsvariablen sind nicht zentral in einer `.env`-Datei gespeichert, sondern werden kontextspezifisch direkt in den jeweiligen Dateien referenziert oder generiert. Im Frontend wird die Variable `CONTEXT` sowohl im Astro-Konfigurationsfile (`frontend/astro.config.mjs`) als auch in Layouts und der `robots.txt` gelesen, um Build-Zeiten oder Deployment-Stadien zu steuern. Für die E-Mail-Funktionalität werden die SMTP-Parameter wie Host, Port, Benutzername und Passwort sowie das Ziel-E-Mail-Adressat (`CONTACT_TO`) direkt im Code der Funktion `frontend/netlify/functions/contact.mjs` definiert. Diese Werte

fließen in den Serverless-Funktionen ein, um über Nodemailer E-Mails zu versenden. Parallel dazu werden für die Testumgebung spezifische Variablen wie CI, BASELINE_ROOT, TEST_ROOT und PORT in den Playwright- und Testserver-Konfigurationen (playwright.config.ts, tests/server.mjs) gesetzt, um die Automatisierungstests an die jeweilige Laufzeit anzupassen.

Im Backend finden sich Python-basierte Komponenten wie der Mailer (backend/mailer.py) und der Server (backend/server.py), deren Abhängigkeiten in requirements.txt und Testkonfigurationen (pytest.ini) festgelegt sind. Die Cron-Jobs zur automatisierten Webhook-Auslösung sind im Verzeichnis .emergent/cron organisiert, wo Skripte wie watch_crons.sh und dispatch_webhook.sh die Ausführung steuern. Diese Struktur trennt die produktive Konfiguration von den Testdaten und Automatisierungsskripten klar voneinander, wobei sensible Daten wie SMTP-Passwörter nur dort definiert sind, wo sie benötigt werden, und nicht global im Repository verteilt sind.

Kurzreferenz

Siehe CONFIGURATION.md.

10. Dependencies

Erklärung

Abhängigkeiten stammen aus package.json; tatsächliche Nutzung zeigt sich in Import-Facts.

Keine Dependencies belegt.

Beobachtete Imports (Auszug): frontend/astro.config.mjs !' astro/config; frontend/astro.config.mjs !' node:fs/promises; frontend/netlify/functions/contact.mjs !' nodemailer; frontend/playwright.config.ts !' @playwright/test; frontend/src/components/ContactCTA.astro !' ../data/site; frontend/src/components/ContactCTA.astro !' ../Icon.astro; frontend/src/components/ContactForm.astro !' ../data/site; frontend/src/components/ContactForm.astro !' ../Icon.astro; frontend/src/components/FloatingWhatsApp.astro !' ../data/site; frontend/src/components/FloatingWhatsApp.astro !' ../Icon.astro; frontend/src/components/Footer.astro !' ../data/site; frontend/src/components/Footer.astro !' ../Logo.astro.

Kurzreferenz

Siehe DEPENDENCIES.md.

11. Build und lokale Entwicklung

Erklärung

Das Deployment dieses Repositories folgt einem klar definierten, aber teilweise noch nicht vollständig dokumentierten Prozess, der sich primär um die Astro-Frontend-Modernisierung dreht. Der Build-Prozess wird auf Netlify ausgelöst, wobei dort in der Konfiguration (netlify.toml) explizit npm run build als Befehl hinterlegt ist. Das Ergebnis dieses Builds befindet sich im Verzeichnis dist, welches als Publish-Target für die Veröffentlichung dient. Für die Bündelung der Assets innerhalb dieses Build-Prozesses wird der Node.js-Bundler esbuild verwendet, was auf eine Optimierung für Geschwindigkeit und Größe hindeutet.

Parallel zum Frontend-Build läuft ein spezifischer Workflow für Serverless-Funktionen. Im Pfad frontend/netlify/functions/contact.mjs ist eine Funktion definiert, die über die Route /api/contact erreichbar ist. Diese Funktion wird im Deployment-Kontext so umgewandelt, dass sie unter der URL-Struktur /.netlify/functions/:splat abgerufen werden kann. Wenn eine Anfrage auf /api/contact eingeht, wird diese an den spezifischen Serverless-Funktionpfad weitergeleitet und dort verarbeitet. Innerhalb dieser Logik ist die Absendung von E-Mails integriert; der Code greift hierbei auf einen SMTP-Server zu, um Benachrichtigungen auszulösen, was eine direkte Kopplung zwischen der Frontend-Anwendung und externen Mailediensten darstellt.

Die Sicherheitseinstellungen für das Deployment sind in der Netlify-Konfiguration detailliert vorgegeben und gelten für alle Ressourcen (*). Standardmäßig werden sensible Header wie X-Content-Type-Options auf nosniff gesetzt, um MIME-Spoofing zu verhindern, während X-Frame-Options auf DENY festgelegt ist, um Clickjacking-Angriffe zu blockieren. Zusätzlich wird die Referer-Policy auf strict-origin-when-cross-origin eingeschränkt und Zugriff auf sensitive Gerätefunktionen wie Kamera, Mikrofon oder Geolocation via Permissions-Policy explizit deaktiviert. Für statische Assets im Pfad /fonts/* ist eine aggressive Caching-Strategie konfiguriert (public, max-age=31536000, immutable), um die Performance zu maximieren, während diese Header für andere Pfade nicht gecacht werden. <!-- not-evidenced: MIME -->

Es gibt jedoch Bereiche im Repository, die im Kontext des Deployment-Prozesses noch nicht vollständig dokumentiert sind oder deren technische Details fehlen. Beispielsweise ist der genaue Mechanismus, wie die Skripte im Verzeichnis .emergent/cron (wie watch_crons.sh oder dispatch_webhook.sh) in den eigentlichen Deploymentschritt integriert werden, nicht durch die bereitgestellten Fakten belegt. Auch die spezifischen Umgebungsvariablen für den SMTP-Server (smtp_57) oder die genauen Abhängigkeiten des Backend-Python-Codes (backend/mailer.py, requirements.txt) sind im Rahmen der Deployment-

Konfiguration nicht explizit beschrieben, obwohl sie existieren. Die Datei `.bootstrap-complete` deutet zwar auf einen initialisierten Zustand hin, liefert aber keine Details darüber, wann genau dieser Zustand während eines neuen Deploys validiert wird.

Kurzreferenz

- Netlify build command: `{"command":"npm run build"}`
- Netlify node bundler: `{"bundler":"esbuild"}`

12. Deployment und CI/CD

Erklärung

Das Deployment dieses Repositories folgt einem klar definierten, aber teilweise noch nicht vollständig dokumentierten Prozess, der sich primär um die Astro-Frontend-Modernisierung dreht. Der Build-Prozess wird auf Netlify ausgelöst, wobei dort in der Konfiguration (`netlify.toml`) explizit `npm run build` als Befehl hinterlegt ist. Das Ergebnis dieses Builds befindet sich im Verzeichnis `dist`, welches als Publish-Target für die Veröffentlichung dient. Für die Bündelung der Assets innerhalb dieses Build-Prozesses wird der Node.js-Bundler `esbuild` verwendet, was auf eine Optimierung für Geschwindigkeit und Größe hindeutet.

Parallel zum Frontend-Build läuft ein spezifischer Workflow für Serverless-Funktionen. Im Pfad `frontend/netlify/functions/contact.mjs` ist eine Funktion definiert, die über die Route `/api/contact` erreichbar ist. Diese Funktion wird im Deployment-Kontext so umgewandelt, dass sie unter der URL-Struktur `/.netlify/functions/:splat` abgerufen werden kann. Wenn eine Anfrage auf `/api/contact` eingeht, wird diese an den spezifischen Serverless-Funktionspfad weitergeleitet und dort verarbeitet. Innerhalb dieser Logik ist die Absendung von E-Mails integriert; der Code greift hierbei auf einen SMTP-Server zu, um Benachrichtigungen auszulösen, was eine direkte Kopplung zwischen der Frontend-Anwendung und externen Mailediensten darstellt.

Die Sicherheitseinstellungen für das Deployment sind in der Netlify-Konfiguration detailliert vorgegeben und gelten für alle Ressourcen (`/*`). Standardmäßig werden sensible Header wie `X-Content-Type-Options` auf `nosniff` gesetzt, um MIME-Spoofing zu verhindern, während `X-Frame-Options` auf `DENY` festgelegt ist, um Clickjacking-Angriffe zu blockieren. Zusätzlich wird die Referer-Policy auf `strict-origin-when-cross-origin` eingeschränkt und Zugriff auf sensitive Gerätefunktionen wie Kamera, Mikrofon oder Geolocation via Permissions-Policy explizit deaktiviert. Für statische Assets im Pfad `/fonts/*` ist eine aggressive Caching-Strategie konfiguriert (`public, max-age=31536000, immutable`), um die Performance zu maximieren, während diese Header für andere Pfade nicht gecacht werden. `<!-- not-evidenced: MIME -->`

Es gibt jedoch Bereiche im Repository, die im Kontext des Deployment-Prozesses noch nicht vollständig dokumentiert sind oder deren technische Details fehlen. Beispielsweise ist der genaue Mechanismus, wie die Skripte im Verzeichnis `.emergent/cron` (wie `watch_crons.sh` oder `dispatch_webhook.sh`) in den eigentlichen Deploymentsschritt integriert werden, nicht durch die bereitgestellten Fakten belegt. Auch die spezifischen Umgebungsvariablen für den SMTP-Server (`smtp_57`) oder die genauen Abhängigkeiten des Backend-Python-Codes (`backend/mailer.py`, `requirements.txt`) sind im Rahmen der Deployment-Konfiguration nicht explizit beschrieben, obwohl sie existieren. Die Datei `.bootstrap-complete` deutet zwar auf einen initialisierten Zustand hin, liefert aber keine Details darüber, wann genau dieser Zustand während eines neuen Deploys validiert wird.

Kurzreferenz

Siehe `DEPLOYMENT.md`.

13. Tests und Qualitätssicherung

Erklärung

Gefundene Test-/Spec-Dateien: `frontend/tests/e2e/site.spec.ts`, `frontend/tests/unit/contact.test.ts`, `frontend/tests/unit/function.test.mjs`, `frontend/tests/unit/parity.test.ts`, `frontend/tests/unit/seasons.test.ts`, `tests/__init__.py`.

Kurzreferenz

Siehe `TESTING.md`.

14. Security-relevante Architektur

Erklärung

Die Sicherheitsarchitektur in diesem Repository konzentriert sich primär auf die Validierung und Filterung von Eingabedaten an der Schnittstelle zur Kontaktformular-Funktion, implementiert im Dateipfad `frontend/netlify/functions/contact.mjs`. Als erfahrener Entwickler musst du verstehen, dass hier eine mehrstufige

Verteidigungslinie aufgebaut ist, um das Backend vor Missbrauch zu schützen. Der erste Schutzmechanismus ist die Begrenzung der Anfragegröße; jede POST-Anfrage an den Endpunkt `/api/contact` wird auf maximal 20.000 Bytes geprüft. Wird diese Schwelle überschritten, wird sofort ein HTTP-Statuscode 413 (Request Entity Too Large) zurückgegeben, was verhindert, dass das System durch extrem große Payloads überlastet oder zum Absturz gebracht wird.

Parallel dazu erfolgt eine strikte Sanitierung der Nutzereingaben, um Cross-Site Scripting (XSS)-Angriffe abzuwehren. Im Code an Zeile 47 wird explizit `escapeHtml` auf `true` gesetzt, während das Entfernen von Tags (`stripTags`) deaktiviert ist und die Prüfung unsicherer Header (`headerUnsafe`) ebenfalls ausbleibt. Dies bedeutet, dass HTML-Tags nicht entfernt werden, aber alle darin enthaltenen Skripte oder gefährlichen Entitäten vor der Verarbeitung sicherheitstechnisch neutralisiert werden. Sollte ein Angreifer versuchen, diese Logik zu umgehen oder ungültige Daten zu senden, reagiert die Funktion mit einem HTTP-Statuscode 400 (Bad Request), was eine klare Signalisierung für fehlerhafte oder nicht konforme Eingaben darstellt.

Zusätzlich zum Schutz vor direkten Angriffen ist ein Honeypot integriert, der sich in den Feldern des Formulars befindet, um automatisierte Bots zu erkennen und abzuwehren. Wenn ein Skript versucht, das unsichtbare Feld `website` auszufüllen, wird dies erkannt; die Logik an Zeile 74 sorgt dafür, dass bei diesem Vorfall kein stiller Erfolg (`silentSuccess: false`) zurückgegeben wird, sondern der Prozess unterbrochen oder als verdächtig markiert wird. Im Falle von Systemfehlern oder Verbindungsproblemen im Backend reagiert die Funktion robust mit Statuscodes 502 (Bad Gateway) und 503 (Service Unavailable), was sicherstellt, dass Fehlerzustände transparent kommuniziert werden, ohne dass sensible Daten ausgesetzt werden.

Im Hintergrund wird das gesäuberte Formular-Datenset an den Endpunkt `/api/contact` gesendet, wo es weiterverarbeitet wird. Die erfolgreiche Verarbeitung führt schließlich zum Aufruf einer Mail-Funktion, die über `backend/mailer.py` und `backend/server.py` realisiert ist und eine E-Mail via SMTP versendet. Dieser gesamte Ablauf zeigt ein Design-Bild, bei dem Sicherheit nicht als nachträglicher Gedanke, sondern als integraler Bestandteil der Datenfluss-Logik verankert ist: Von der Größe der Anfrage über die Entschärfung von Inhalten bis hin zur Erkennung automatisierter Angriffe wird jede Ebene des Requests geprüft, bevor eine E-Mail an den Empfänger gesendet wird.

Kurzreferenz

Siehe `SECURITY.md`.

15. belegbare Architekturentscheidungen

Erklärung

Die Anwendung gliedert sich in Astro-Seiten/Komponenten unter `src/` und optionale Serverless-Funktionen unter `Netlify`.

Extrahierte Komponenten (Auszug): `frontend/src/components/ContactCTA.astro`, `frontend/src/components/ContactForm.astro`, `frontend/src/components/FloatingWhatsApp.astro`, `frontend/src/components/Footer.astro`, `frontend/src/components/Header.astro`, `frontend/src/components/Icon.astro`, `frontend/src/components/LegalSection.astro`, `frontend/src/components/Logo.astro`, `frontend/src/components/Rating.astro`, `frontend/src/components/SeasonalServices.astro`, `frontend/src/layouts/BaseLayout.astro`, `frontend/src/layouts/LegalLayout.astro`.

Seiten-Routen ergeben sich aus `src/pages/`, u. a. `frontend/src/pages/404.astro`, `/frontend/src/pages/404`, `frontend/src/pages/agb.astro`, `/frontend/src/pages/agb`, `frontend/src/pages/datenschutz.astro`, `/frontend/src/pages/datenschutz`, `frontend/src/pages/impressum.astro`, `/frontend/src/pages/impressum`, `frontend/src/pages/index.astro`, `/frontend/src/pages/`.

Serverless-Einstiege: `frontend/netlify/functions/contact.mjs`.

Zentrale Laufzeitketten sind als Relationships hinterlegt (siehe Datenflüsse), z. B. `UI ! HTTP ! Rewrite ! Function ! SMTP`.

Kurzreferenz

`_` Nicht dokumentiert — keine belegten Einträge.

16. bekannte Einschränkungen

Erklärung

Die Konfiguration in diesem Repository ist stark durch die Nutzung von Netlify und spezifischen Astro-Plugins geprägt, wobei Sicherheitsrichtlinien und Routing-Logik zentral gesteuert werden. Im Kern der Deployment-Konfiguration befindet sich die Datei `netlify.toml`, welche die HTTP-Antworten für alle Requests (/ *) standardisiert absichert. Dort sind explizit die Header `X-Content-Type-Options` mit dem Wert `nosniff`, `X-Frame-Options` auf `DENY` sowie `Referrer-Policy` auf `strict-origin-when-cross-origin` definiert, um Cross-Site-Scripting und andere Angriffe zu verhindern. Zusätzlich wird die Nutzung sensibler APIs wie Kamera, Mikrofon und Geolokation über den Header `Permissions-Policy` blockiert (`camera=()`, `microphone=()`).

geolocation=()). Für statische Assets, insbesondere Schriftarten unter dem Pfad /fonts/, ist eine aggressive Caching-Strategie konfiguriert: Der Header Cache-Control erlaubt ein öffentliches Caching mit einer Gültigkeitsdauer von einem Jahr (max-age=31536000) und der Kennzeich

Das Routing wird in derselben netlify.toml durch Rewrite-Regeln und Redirects definiert. Alle Anfragen unter /api/* werden transparent an die Serverless-Funktionen weitergeleitet, wobei der Pfadteil hinter dem Slash als Parameter für die Funktion /.netlify/functions/:splat genutzt wird und eine Statuscode 200 zurückgibt. Fehlt ein passender Treffer im Frontend oder in den Funktionen, leitet das System standardmäßig auf /404.html weiter, was durch einen Redirect mit dem Status 404 realisiert wird. Diese Struktur ermöglicht es, die API-Endpunkte dynamisch zu erweitern, ohne dass diese im Frontend-Code explizit als statische Pfade gehärtet werden müssen.

Die Umgebungsvariablen sind nicht zentral in einer .env-Datei gespeichert, sondern werden kontextspezifisch direkt in den jeweiligen Dateien referenziert oder generiert. Im Frontend wird die Variable CONTEXT sowohl im Astro-Konfigurationsfile (frontend/astro.config.mjs) als auch in Layouts und der robots.txt gelesen, um Build-Zeiten oder Deployment-Stadien zu steuern. Für die E-Mail-Funktionalität werden die SMTP-Parameter wie Host, Port, Benutzername und Passwort sowie das Ziel-E-Mail-Adressat (CONTACT_TO) direkt im Code der Funktion frontend/netlify/functions/contact.mjs definiert. Diese Werte fließen in den Serverless-Funktionen ein, um über Nodemailer E-Mails zu versenden. Parallel dazu werden für die Testumgebung spezifische Variablen wie CI, BASELINE_ROOT, TEST_ROOT und PORT in den Playwright- und Testserver-Konfigurationen (playwright.config.ts, tests/server.mjs) gesetzt, um die Automatisierungstests an die jeweilige Laufzeit anzupassen.

Im Backend finden sich Python-basierte Komponenten wie der Mailer (backend/mailer.py) und der Server (backend/server.py), deren Abhängigkeiten in requirements.txt und Testkonfigurationen (pytest.ini) festgelegt sind. Die Cron-Jobs zur automatisierten Webhook-Auslösung sind im Verzeichnis .emergent/cron organisiert, wo Skripte wie watch_crons.sh und dispatch_webhook.sh die Ausführung steuern. Diese Struktur trennt die produktive Konfiguration von den Testdaten und Automatisierungsskripten klar voneinander, wobei sensible Daten wie SMTP-Passwörter nur dort definiert sind, wo sie benötigt werden, und nicht global im Repository verteilt sind.

Kurzreferenz

Nicht dokumentiert — keine belegten Einträge.

17. belegbarer Technical Debt

Erklärung

Dokumentierter Commit (Analyse-HEAD): 3c837baf80be192e4c782180ccb951126dce9878 auf Branch feat/astro-modernisierung, erzeugt 2026-09-09T18:11:01.553Z.

Inkonsistenzen und getrennte Anforderungsquellen (Node engines vs README, package vs lock Version) sind unter Konfiguration als Facts geführt — nicht als spekulativer Technical Debt.

Kurzreferenz

Nicht dokumentiert — keine belegten Einträge.

18. Git-/Änderungsverlauf

Erklärung

Dokumentierter Commit (Analyse-HEAD): 3c837baf80be192e4c782180ccb951126dce9878 auf Branch feat/astro-modernisierung, erzeugt 2026-09-09T18:11:01.553Z.

Inkonsistenzen und getrennte Anforderungsquellen (Node engines vs README, package vs lock Version) sind unter Konfiguration als Facts geführt — nicht als spekulativer Technical Debt.

Kurzreferenz

Siehe GIT_HISTORY.md. HEAD 3c837baf80be192e4c782180ccb951126dce9878.

19. aktueller technischer Zustand

Erklärung

Das Repository für das Projekt „Projektübersicht“ befindet sich aktuell im Branch feat/astro-modernisierung und ist als moderne Astro-Anwendung strukturiert, die auf Netlify bereitgestellt wird. Der Build-Prozess wird durch den Befehl npm run build in der Datei netlify.toml ausgelöst, wobei das resultierende Ausgabeverzeichnis dist direkt an Netlify gepusht wird. Für die Bündelung des Frontends steht esbuild als Node-Bundler zur Verfügung. Die Anwendung folgt einer klaren Komponentenarchitektur: Das Layout wird durch BaseLayout und den speziellen LegalLayout definiert, während UI-Elemente wie Header, Footer,

Logo, Icon sowie spezifische Sektionen wie ContactCTA, ContactForm, FloatingWhatsApp, SeasonalServices und Rating modular aufgebaut sind. Die Seitenstruktur umfasst die Startseite (index), Unterseiten für Leistungen (leistungen) und Team (team) sowie rechtliche Pflichtseiten wie agb, datenschutz und impressum. Zusätzlich existiert eine eigene Fehlerseite (404).

Die Sicherheitskonfiguration ist in der netlify.toml detailliert festgelegt. Für alle Ressourcen (/*) werden die Header X-Content-Type-Options mit dem Wert nosniff, X-Frame-Options mit DENY sowie Referrer-Policy auf strict-origin-when-cross-origin gesetzt, um Clickjacking und MIME-Typ-Missbrauch zu verhindern. Zudem wird die Nutzung sensibler Gerätefunktionen wie Kamera, Mikrofon und Geolokalisierung durch das Header Permissions-Policy explizit blockiert (camera=(), microphone=(), geolocation=()). Für Font-Ressourcen unter dem Pfad /fonts/* ist eine aggressive Caching-Strategie konfiguriert (Cache-Control: public, max-age=31536000, immutable), um die Performance zu optimieren. <!-- not-evidenced: MIME -->

Die Interaktion mit dem Backend erfolgt über einen spezifischen Endpunkt für Kontaktanfragen. Die Funktion contact.mjs im Verzeichnis frontend/netlify/functions greift auf den Route-Pfad /api/contact zu und erlaubt ausschließlich HTTP-POST-Anfragen; alle anderen Methoden werden mit einem 405-Statuscode abgewiesen. Wenn ein Formular (ContactForm) übermittelt wird, wird diese Anfrage an diesen Endpunkt gesendet. Im Hintergrund verarbeitet das Python-Skript backend/mailer.py, das über server.py erreichbar ist, die eingehenden Daten und leitet sie per SMTP (unterstützt durch Nodemailer) weiter, um E-Mails zu versenden. Zusätzlich gibt es eine Cron-Job-Infrastruktur im Ordner .emergent/cron, die Webhooks überwacht und dispatchet, was auf eine geplante Automatisierung hindeutet.

Kurzreferenz

1234 Fakten / 280 Beziehungen · 2026-09-09T18:11:01.553Z.

